

Funciones XPath

Introducción

XPath proporciona un conjunto amplio de funciones integradas que permiten manipular y procesar datos de manera eficiente. Estas funciones se clasifican en diferentes categorías según su propósito: funciones numéricas, de cadena, booleanas, de secuencias, de nodos, etc.

En XPath 2.0 y versiones posteriores, el conjunto de funciones se amplió considerablemente, proporcionando mayor potencia y flexibilidad para trabajar con documentos XML.

Funciones Numéricas

abs()

Sintaxis: `abs($arg as numeric?) as numeric?`

Devuelve el **valor absoluto** de un número.

Ejemplo:

```
abs(-5)           <!-- Resultado: 5 -->
abs(3.14)         <!-- Resultado: 3.14 -->
abs(-10.5)        <!-- Resultado: 10.5 -->
```



Uso práctico:

```
<!-- Seleccionar productos con descuento mayor a 10 en valor -->
//producto[abs(@descuento) > 10]
```



ceiling()

Sintaxis: `ceiling($arg as numeric?) as numeric?`

Redondea un número hacia arriba al entero más cercano.

Ejemplo:

```
ceiling(3.2)      <!-- Resultado: 4 -->
ceiling(-3.8)     <!-- Resultado: -3 -->
ceiling(5)        <!-- Resultado: 5 -->
```



Uso práctico:

```
<!-- Calcular páginas necesarias (20 items por página) -->
ceiling(count(//item) div 20)
```



floor()

Sintaxis: `floor($arg as numeric?) as numeric?`

Redondea un número hacia abajo al entero más cercano.

Ejemplo:

```
floor(3.8)        <!-- Resultado: 3 -->
floor(-3.2)       <!-- Resultado: -4 -->
floor(5)          <!-- Resultado: 5 -->
```



Uso práctico:

```
<!-- Obtener la parte entera de un precio -->
floor(//producto/@precio)
```



max()

Sintaxis: `max($arg as xdt:anyAtomicType*) as xdt:anyAtomicType?`

Devuelve el valor máximo de una secuencia.

Ejemplo:

```
max((1, 5, 3, 9, 2))           <!-- Resultado: 9 -->
max(//producto/@precio)       <!-- Precio máximo -->
max((10, 20, 30))             <!-- Resultado: 30 -->
```

Uso práctico:

```
<!-- Seleccionar productos con el precio máximo -->
//producto[@precio = max(//producto/@precio)]
```

min()

Sintaxis: `min($arg as xdt:anyAtomicType*) as xdt:anyAtomicType?`

Devuelve el valor mínimo de una secuencia.

Ejemplo:

```
min((1, 5, 3, 9, 2))           <!-- Resultado: 1 -->
min(//producto/@precio)       <!-- Precio mínimo -->
```

Uso práctico:

```
<!-- Encontrar el precio más bajo en cada categoría -->
//categoria/min(producto/@precio)
```

sum()

Sintaxis: `sum($arg as xdt:anyAtomicType*) as xdt:anyAtomicType`

Calcula la suma de una secuencia de valores.

Ejemplo:

```
sum((1, 2, 3, 4, 5))
sum(//producto/@precio)
sum(//pedido/total)
```

```
<!-- Resultado: 15 -->
<!-- Suma de todos los precios -->
<!-- Total de todos los pedidos -->
```

Uso práctico:

```
<!-- Calcular el valor total del inventario -->
sum(//producto/@precio * @stock)
```

avg()

Sintaxis: `avg($arg as xdt:anyAtomicType*) as xdt:anyAtomicType?`

Calcula el promedio de una secuencia de valores.

Ejemplo:

```
avg((1, 2, 3, 4, 5))
avg(//producto/@precio)
avg(//alumno/nota)
```

```
<!-- Resultado: 3 -->
<!-- Precio promedio -->
<!-- Nota media -->
```

Uso práctico:

```
<!-- Seleccionar alumnos con nota superior a la media -->
//alumno[nota > avg(//alumno/nota)]
```

Funciones de Cadena

string()

Sintaxis: `string($arg as item())? as xs:string`

Convierte un valor a cadena de texto.

Ejemplo:

```
string(123)
string(//libro/titulo)
string(true())
```

```
<!-- Resultado: "123" -->
<!-- Convierte el título a str
<!-- Resultado: "true" -->
```

Uso práctico:

```
<!-- Obtener el texto de un nodo -->
string(//parrafo[1])
```

concat()

Sintaxis: `concat($arg1 as xs:anyAtomicType?, $arg2 as xs:anyAtomicType?, ... ) as xs:string`

Concatena dos o más cadenas de texto.

Ejemplo:

```
concat("Hola", " ", "Mundo")
concat(//persona/nombre, " ", //persona/apellido)
concat("Precio: ", @precio, "€")
```

```
<!-- Resultado: "Hola " -->
```

Uso práctico:

```
<!-- Crear nombre completo -->
concat(//empleado/nombre, " ", //empleado/apellido1, " ", //empleado/a
```

contains()

Sintaxis: `contains($arg1 as xs:string?, $arg2 as xs:string?) as xs:boolean`

Comprueba si una cadena contiene otra cadena.

Ejemplo:

```
contains("Hola Mundo", "Mundo")      <!-- Resultado: true -->
contains("ejemplo", "xyz")            <!-- Resultado: false -->
```

Uso práctico:

```
<!-- Buscar libros que contengan "Java" en el título -->
//libro[contains(titulo, "Java")]

<!-- Buscar emails de Gmail -->
//contacto[contains(email, "@gmail.com")]
```

starts-with()

Sintaxis: `starts-with($arg1 as xs:string?, $arg2 as xs:string?) as xs:boolean`

Comprueba si una cadena comienza con otra cadena específica.

Ejemplo:

```
starts-with("Hola Mundo", "Hola")    <!-- Resultado: true -->
starts-with("ejemplo", "ex")          <!-- Resultado: true -->
```

Uso práctico:

```
<!-- Seleccionar productos cuyo código empiece por "A" -->
//producto[starts-with(@codigo, "A")]

<!-- Buscar URLs que empiecen por https -->
//enlace[starts-with(@href, "https://")]
```

ends-with()

Sintaxis: `ends-with($arg1 as xs:string?, $arg2 as xs:string?) as xs:boolean`

Comprueba si una cadena termina con otra cadena específica.

Ejemplo:

```
ends-with("archivo.xml", ".xml")      <!-- Resultado: true -->
ends-with("documento.pdf", ".doc")    <!-- Resultado: false -->
```

Uso práctico:

```
<!-- Seleccionar archivos XML -->
//archivo[ends-with(@nombre, ".xml")]

<!-- Buscar emails con dominio .es -->
//contacto[ends-with(email, ".es")]
```

string-length()

Sintaxis: `string-length($arg as xs:string?) as xs:integer`

Devuelve la longitud de una cadena.

Ejemplo:

```
string-length("Hola")      <!-- Resultado: 4 -->
string-length("")          <!-- Resultado: 0 -->
string-length(//libro/titulo) <!-- Longitud del título -->
```

Uso práctico:

```
<!-- Seleccionar comentarios largos (más de 100 caracteres) -->
//comentario[string-length(.) > 100]
```

```
<!-- Validar códigos postales (5 dígitos) -->
//direccion[string-length(codigo_postal) = 5]
```

substring()

Sintaxis: `substring($sourceString as xs:string?, $start as xs:double, $length as xs:double?) as xs:string`

Extrae una subcadena de una cadena.

Ejemplo:

<code>substring("Hola Mundo", 1, 4)</code>	<code><!-- Resultado: "Hola" --></code> 
<code>substring("Hola Mundo", 6)</code>	<code><!-- Resultado: "Mundo" --></code>
<code>substring("ejemplo", 2, 3)</code>	<code><!-- Resultado: "jem" --></code>

Uso práctico:

```
<!-- Obtener los primeros 3 caracteres de un código -->
substring(//producto/@codigo, 1, 3)

<!-- Extraer el año de una fecha (formato YYYY-MM-DD) -->
substring(//pedido/@fecha, 1, 4)
```



normalize-space()

Sintaxis: `normalize-space($arg as xs:string?) as xs:string`

Elimina espacios en blanco iniciales y finales, y reduce múltiples espacios a uno solo.

Ejemplo:

<code>normalize-space(" Hola Mundo ")</code>	<code><!-- Resultado: "Hola" --></code> 
<code>normalize-space(" texto ")</code>	<code><!-- Resultado: "texto" --></code>

Uso práctico:

```
<!-- Limpiar espacios en nombres -->
//persona[normalize-space(nombre) = "Juan"]
```



upper-case()

Sintaxis: `upper-case($arg as xs:string?) as xs:string`

Convierte una cadena a mayúsculas.

Ejemplo:

```
upper-case("hola")
upper-case("Ejemplo 123")
```

```
<!-- Resultado: "HOLA" -->
<!-- Resultado: "EJEMPLO 123" -->
```



lower-case()

Sintaxis: `lower-case($arg as xs:string?) as xs:string`

Convierte una cadena a minúsculas.

Ejemplo:

```
lower-case("HOLA")
lower-case("Ejemplo 123")
```

```
<!-- Resultado: "hola" -->
<!-- Resultado: "ejemplo 123" -->
```



Uso práctico:

```
<!-- Búsqueda sin distinción de mayúsculas/minúsculas -->
//libro[lower-case(titulo) = "el quijote"]
```



replace()

Sintaxis: `replace($input as xs:string?, $pattern as xs:string, $replacement as xs:string) as xs:string`

Reemplaza partes de una cadena usando expresiones regulares.

Ejemplo:

```
replace("2024-01-15", "-", "/")
replace("abc123", "\d", "X")
```

```
<!-- Resultado: "2024" -->
<!-- Resultado: "abcXXX" -->
```

Uso práctico:

```
<!-- Formatear números de teléfono -->
replace(//contacto/telefono, "(\d{3})(\d{3})(\d{3})", "$1-$2-$3")
```

matches()

Sintaxis: `matches($input as xs:string?, $pattern as xs:string) as xs:boolean`

Comprueba si una cadena coincide con una expresión regular.

Ejemplo:

```
matches("abc123", "^\\w+\\d+$")
matches("email@dominio.com", ".*@.*")
```

```
<!-- Resultado: true -->
<!-- Resultado: true -->
```

Uso práctico:

```
<!-- Validar formato de email -->
//contacto[matches(email, "^\\w.-]+@[\\w.-]+\\.\\w+$")]

<!-- Validar DNI español -->
//persona[matches(dni, "^\\d{8}[A-Z]$")]
```

Funciones Booleanas

boolean()

Sintaxis: `boolean($arg as item(*)*) as xs:boolean`

Convierte un valor a booleano.

Ejemplo:

<code>boolean(1)</code>	<code><!-- Resultado: true --></code>
<code>boolean(0)</code>	<code><!-- Resultado: false --></code>
<code>boolean("")</code>	<code><!-- Resultado: false --></code>
<code>boolean("texto")</code>	<code><!-- Resultado: true --></code>

true()

Sintaxis: `true() as xs:boolean`

Devuelve el valor booleano verdadero.

Ejemplo:

<code>true()</code>	<code><!-- Resultado: true --></code>
---------------------	---

Uso práctico:

```
<!-- Seleccionar elementos activos -->  
//producto[@activo = true()]
```

false()

Sintaxis: `false() as xs:boolean`

Devuelve el valor booleano falso.

Ejemplo:

```
false()
```

```
<!-- Resultado: false -->
```



not()

Sintaxis: `not($arg as item()*) as xs:boolean`

Niega un valor booleano.

Ejemplo:

```
not(true())
```

```
not(false())
```

```
not(//producto[@descuento])
```

```
<!-- Resultado: false -->
```

```
<!-- Resultado: true -->
```

```
<!-- true si no hay descuento
```



Uso práctico:

```
<!-- Seleccionar productos sin stock -->
```

```
//producto[not(@stock) or @stock = 0]
```

```
<!-- Libros que no tienen autor -->
```

```
//libro[not(autor)]
```



Funciones de Secuencias

count()

Sintaxis: `count($arg as item()*) as xs:integer`

Cuenta el número de elementos en una secuencia.

Ejemplo:



```
count((1, 2, 3, 4, 5))  
count(//libro)  
count(//producto[@stock > 0])
```

```
<!-- Resultado: 5 -->  
<!-- Número de libros -->  
<!-- Productos con stock -->
```

Uso práctico:

```
<!-- Categorías con más de 10 productos -->  
//categoria[count(producto) > 10]
```



position()

Sintaxis: `position()` as `xs:integer`

Devuelve la posición del nodo actual en el contexto.

Ejemplo:

```
//libro[position() = 1]  
//libro[position() < 5]  
//libro[position() mod 2 = 0]
```

```
<!-- Primer libro -->  
<!-- Primeros 4 libros -->  
<!-- Libros en posiciones pares -->
```



Uso práctico:

```
<!-- Seleccionar los primeros 3 elementos -->  
//producto[position() <= 3]
```



last()

Sintaxis: `last()` as `xs:integer`

Devuelve la posición del último nodo en el contexto.

Ejemplo:



```
//libro[last()]           <!-- Último libro -->
//libro[position() = last()] <!-- Último libro (equivalente) -->
//producto[position() > last() - 3] <!-- Últimos 3 productos -->
```

empty()

Sintaxis: `empty($arg as item(*) as xs:boolean)`

Comprueba si una secuencia está vacía.

Ejemplo:

```
empty()           <!-- Resultado: true -->
empty((1, 2, 3))  <!-- Resultado: false -->
empty(//libro[precio > 100]) <!-- true si no hay libros caros -->
```

Uso práctico:

```
<!-- Verificar si hay productos descatalogados -->
not(empty(//producto[@descatalogado = true()]))
```

exists()

Sintaxis: `exists($arg as item(*) as xs:boolean)`

Comprueba si una secuencia contiene al menos un elemento.

Ejemplo:

```
exists(//libro[@disponible = true()]) <!-- ¿Hay libros disponibles? -->
exists(//pedido[@estado = "pendiente"]) <!-- ¿Hay pedidos pendientes? -->
```

Uso práctico:

```
<!-- Clientes que tienen pedidos -->
//cliente[exists(pedido)]
```

distinct-values()

Sintaxis: `distinct-values($arg as xdt:anyAtomicType*) as xdt:anyAtomicType*`

Devuelve los valores únicos de una secuencia, eliminando duplicados.

Ejemplo:

```
distinct-values((1, 2, 2, 3, 3, 3))      <!-- Resultado: (1, 2) -->
distinct-values(//libro/autor)          <!-- Lista de autores únicos -->
distinct-values(//producto/@categoria) <!-- Categorías únicas -->
```

Uso práctico:

```
<!-- Obtener lista de ciudades sin duplicados -->
distinct-values(//cliente/ciudad)
```

index-of()

Sintaxis: `index-of($seqParam as xdt:anyAtomicType*, $srchParam as xdt:anyAtomicType) as xs:integer*`

Devuelve las posiciones de un valor en una secuencia.

Ejemplo:

```
index-of((1, 2, 3, 2, 4), 2)      <!-- Resultado: (2, 4) -->
index-of(("a", "b", "c"), "b")    <!-- Resultado: 2 -->
```

reverse()

Sintaxis: `reverse($arg as item()*) as item()*`

Invierte el orden de una secuencia.

Ejemplo:

```
reverse((1, 2, 3, 4, 5))           <!-- Resultado: (5, 4, 3, 2, 1) -->
reverse(//libro)                   <!-- Libros en orden inverso -->
```

Uso práctico:

```
<!-- Obtener los últimos 5 productos añadidos -->
reverse(//producto)[position() <= 5]
```

remove()

Sintaxis: `remove($target as item()*, $position as xs:integer) as item()*`

Elimina un elemento en una posición específica de una secuencia.

Ejemplo:

```
remove((1, 2, 3, 4, 5), 3)         <!-- Resultado: (1, 2, 4, 5) -->
remove(//libro, 1)                 <!-- Todos los libros excepto el primero -->
```

Funciones de Nodos

name()

Sintaxis: `name($arg as node()?) as xs:string`

Devuelve el nombre cualificado de un nodo.

Ejemplo:


```
name(//libro[1])
name(/*)
```

```
<!-- Resultado: "libro" -->
<!-- Nombre del elemento raíz -->
```

Uso práctico:

```
<!-- Seleccionar elementos cuyo nombre empiece por "prod" -->
//*[starts-with(name(), "prod")]
```

local-name()

Sintaxis: `local-name($arg as node()?) as xs:string`

Devuelve el nombre local de un nodo (sin prefijo de namespace).

Ejemplo:

```
local-name(//libro[1])
```

```
<!-- Nombre local del elemento -->
```

Uso práctico:

```
<!-- Útil cuando se trabaja con namespaces -->
//*[local-name() = "titulo"]
```

namespace-uri()

Sintaxis: `namespace-uri($arg as node()?) as xs:anyURI`

Devuelve el URI del namespace de un nodo.

Ejemplo:

```
namespace-uri(//libro[1])
```

```
<!-- URI del namespace -->
```

doc()

Sintaxis: `doc($uri as xs:string?) as document-node()?`

Carga un documento XML externo.

Ejemplo:

```
doc("catalogo.xml")//libro           <!-- Libros del documento -->
doc("http://ejemplo.com/datos.xml")//* <!-- Cargar desde URL -->
```

Uso práctico:

```
<!-- Combinar datos de múltiples documentos -->
doc("clientes.xml")//cliente[@id = doc("pedidos.xml")//pedido/@cliente]
```

Ejemplos Prácticos Integrados

Ejemplo 1: Análisis de Biblioteca

```
<biblioteca>
  <libro id="1">
    <titulo>El Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
    <precio>25.50</precio>
    <stock>12</stock>
  </libro>
  <libro id="2">
    <titulo>Cien años de soledad</titulo>
    <autor>Gabriel García Márquez</autor>
    <precio>18.90</precio>
    <stock>8</stock>
  </libro>
  <libro id="3">
    <titulo>1984</titulo>
    <autor>George Orwell</autor>
    <precio>15.75</precio>
    <stock>0</stock>
  </libro>
</biblioteca>
```

```
</libro>
</biblioteca>
```

Consultas útiles:

```
<!-- Libros disponibles (con stock) -->
//libro[exists(@stock) and @stock > 0]

<!-- Precio promedio -->
avg(//libro/precio)

<!-- Libro más caro -->
//libro[precio = max(//libro/precio)]

<!-- Títulos en mayúsculas -->
//libro/upper-case(titulo)

<!-- Contar libros por autor -->
count(//libro[autor = "Gabriel García Márquez"])

<!-- Valor total del inventario -->
sum(//libro/precio * stock)
```

Ejemplo 2: Gestión de Pedidos

```
<pedidos>
  <pedido id="P001" fecha="2024-01-15">
    <cliente email="juan@ejemplo.com">Juan Pérez</cliente>
    <total>125.50</total>
    <estado>entregado</estado>
  </pedido>
  <pedido id="P002" fecha="2024-01-16">
    <cliente email="maria@ejemplo.com">María López</cliente>
    <total>78.25</total>
    <estado>pendiente</estado>
  </pedido>
</pedidos>
```

Consultas útiles:



```
<!-- Pedidos pendientes -->
//pedido[estado = "pendiente"]

<!-- Emails únicos de clientes -->
distinct-values(//cliente/@email)

<!-- Total de ventas -->
sum(//pedido/total)

<!-- Pedidos de enero 2024 -->
//pedido[starts-with(@fecha, "2024-01")]

<!-- Pedidos con importe superior a la media -->
//pedido[total > avg(//pedido/total)]
```